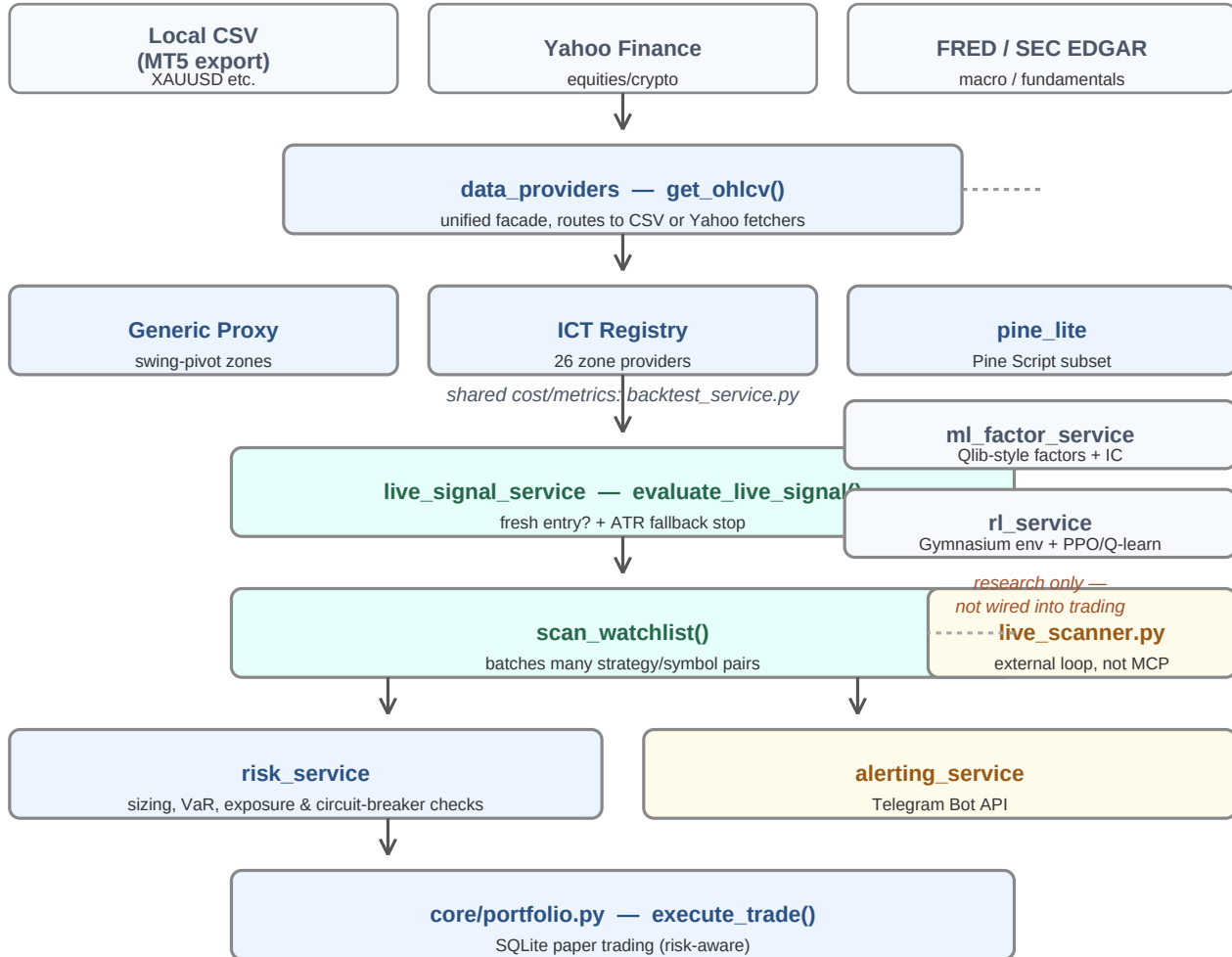


TradingView MCP — System Architecture

3 backtest engines → risk-aware paper trading → live signal scanning → alerting, plus Qlib-style ML and FinRL-style RL research layers. All data local (CSV/Yahoo/FRED/EDGAR) — no live broker connection.



How a Trade Actually Happens

`scripts/live_scanner.py` (running on a timer, outside the MCP request/response loop) → `scan_watchlist()` loops over configured strategy/symbol pairs → `evaluate_live_signal()` re-runs the matching backtest engine (`run_ict_backtest` / `run_custom_backtest` / `run_pine_backtest` — the exact same code used for manual backtesting) over a short trailing window and checks whether its most recent trade opened on the very last bar. If so, it computes a labeled ATR(14) fallback stop (explicitly *not* the strategy's real internal stop, which no engine's trade log exposes) → if the signal is a fresh LONG, `execute_trade()` in `core/portfolio.py` sizes the position via `risk_service.calc_position_size`, gates it through `check_exposure_limits/check_circuit_breaker`, and writes the fill to SQLite → optionally `alerting_service.send_telegram_alert` notifies the user. Short signals are always reported, never auto-traded (the paper portfolio has no short-selling support).

Design Principles

- 1. Keep the LLM out of the execution hot path.** Live scanning, risk checks, execution, and portfolio updates are deterministic scheduled Python (`live_scanner.py`). Claude/MCP stays for judgment-heavy, non-time-critical work: strategy design, optimization, walk-forward review, reporting. Never call an LLM per-symbol per-scan for a real-money decision.
- 2. Reuse, don't duplicate detection logic.** The live signal evaluator, the ML feature builder, and the RL environment all call into the exact same backtest-engine and `indicators_calc.py` code that manual backtesting uses — no separate "live" reimplementations of zone/indicator logic exist anywhere.
- 3. Free/open-source only, with honest fallbacks.** QuantLib (risk math), FRED and SEC EDGAR (data), LightGBM-or-pure-stdlib-logistic-regression (ML), and PPO-or-pure-stdlib-Q-learning (RL) were all built with graceful degradation baked in — several of these heavy/compiled dependencies genuinely failed to load in this session's environment (see "Environment Notes" below), and the system reports which path actually ran rather than silently pretending the primary path succeeded.
- 4. Research output is not trading output.** `run_alpha_factor_analysis` and `train_rl_trading_agent` are diagnostic-only — nothing in `scan_watchlist` consults them. Wiring either into live signal filtering would be a deliberate, separate next step.

"Agents" — the Multi-Agent Debate Pipeline

This is the only literal multi-agent concept in the codebase (pre-existing, in `multi_agent_service.py` — separate from the live trading pipeline above, which uses plain function calls, not agents-in-debate). Three heuristic scorers are combined into one verdict by the `multi_agent_analysis` MCP tool:

File	What it does
Technical agent	Scores momentum/trend from RSI, MACD, and moving averages via <code>compute_metrics / compute_momentum_score</code> (<code>indicators.py</code>).
Sentiment agent	<code>calculate_sentiment_score</code> — heuristic score from price momentum + MACD/RSI alignment, normalized to a -3..+3 scale.
Risk agent	<code>calculate_risk_score</code> — a volatility/drawdown-flavored risk read computed over the same indicator set.

Files Built This Session (Live Trading Pipeline)

File	What it does
<code>risk_service.py</code>	Pure-math risk functions: <code>calc_position_size</code> (risk-based sizing), <code>calc_var</code> (historical/parametric VaR+CVaR), <code>check_exposure_limits</code> , <code>check_circuit_breaker</code> , <code>calc_option_greeks</code> (QuantLib Black-Scholes). No state, no I/O besides QuantLib.
<code>core/portfolio.py</code>	SQLite paper-trading store (built earlier, was unused). This session wired <code>risk_service</code> into <code>execute_trade</code> — risk-based sizing plus exposure/circuit-breaker gating on every BUY.
<code>live_signal_service.py</code>	<code>evaluate_live_signal</code> reuses the 3 backtest engines to detect a fresh entry on the latest bar and computes a fallback ATR stop. <code>scan_watchlist</code> batches this and can auto-execute paper trades.
<code>alerting_service.py</code>	Telegram Bot API wrapper (plain requests, no extra dependency) — <code>send_telegram_alert</code> , <code>format_signal_alert</code> .
<code>scripts/live_scanner.py</code>	Standalone (not MCP) scheduler: a dependency-free sleep loop driving <code>scan_watchlist</code> on an interval, external to the MCP request/response cycle.
<code>data_providers/_init_.py</code>	<code>get_ohlcv</code> — unified facade routing to the existing CSV or Yahoo fetchers.
<code>data_providers/fred_provider.py</code>	FRED (St. Louis Fed) macro series — free, official, needs <code>FRED_API_KEY</code> .
<code>data_providers/edgar_provider.py</code>	SEC EDGAR XBRL company fundamentals — free, official, needs <code>SEC_EDGAR_USER_AGENT</code> .
<code>ml_factor_service.py</code>	Qlib-style alpha research: features from <code>indicators_calc.py</code> , LightGBM-or-logistic model, reports Information Coefficient + hit rate. Research-only, not wired into trading.
<code>rl_service.py</code>	FinRL-style long/flat Gymnasium env; PPO-or-Q-learning agent. Research-only, not wired into trading.

Pre-Existing Files — 3 Backtest Engines + Shared Foundation

File	What it does
server.py	MCP tool registration only — thin delegation, zero business logic. 62 tools total.
custom_strategy_service.py	Generic swing-pivot proxy backtest engine + CSV loader (<code>_fetch_csv_ohlcv</code>) + ICT-registry dispatch (<code>_resolve_ict_provider</code>).
ict_strategies.py	26 hand-coded ICT/SMC zone-provider functions + <code>STRATEGY_REGISTRY</code> .
ict_detectors.py	Reusable detection primitives: liquidity sweeps, order blocks, FVGs, daily/weekly bias, inducement.
pine_lite/ (5 files)	A real constrained Pine Script transpiler: <code>tokenizer.py</code> to <code>parser.py</code> (AST) to <code>validator.py</code> (collects every unsupported construct) to <code>evaluator.py</code> (bar-by-bar execution), plus <code>builtins.py</code> (ta.*math.* functions).
pine_strategy_service.py	Orchestrates <code>pine_lite</code> : parses .txt files, transpiles, simulates trades.
backtest_service.py	Shared cost/metrics math (<code>_apply_costs</code> , <code>_calc_metrics</code>) used by all 3 engines, plus the 9 built-in Yahoo-based indicator strategies (RSI/Bollinger/MACD/EMA-cross/Supertrend/Donchian/etc).
indicators_calc.py	Pure-stdlib indicator math: SMA/EMA/RSI/MACD/ATR/Bollinger/Donchian/Supertrend — no numpy/pandas.
core/errors.py	Structured error envelope (<code>ErrorCode</code> , <code>make_error</code>) used server-wide instead of raw exceptions/strings.
core/types.py	Shared type defs/primitives to avoid circular imports between service modules.

Pre-Existing Files — Market Data / Screener Tools

Unrelated to backtesting; the ~30 pre-existing screener/market-data tools.

File	What it does
screener_service.py	Low-level TradingView data-fetch helpers (trending, Bollinger scan, candle patterns, multi-timeframe).
screener_provider.py	Low-level TradingView screener request plumbing + throttling.
scanner_service.py	Volume breakout / confirmation / smart-volume scanning logic.
stock_screener_service.py	Share-type (common/preferred) stock screening + multi-symbol price lookup.
indicators.py	1600+ lines of TradingView-indicator-derived analytics: support/resistance, market structure detection, stock scoring, Fibonacci — feeds <code>multi_agent_service.py</code> and the screener tools.
multi_agent_service.py	The technical/sentiment/risk "debate" pipeline described on the previous page.
egx_service.py	All Egyptian Exchange (EGX) tools: overview, sector scan/rotation, index analysis, screener, trade plan, Fibonacci.
futures_service.py	CME/COMEX/NYMEX/CBOT futures overview/movers/snapshot/watchlist.
options_service.py	Options chain + unusual volume/OI activity for US stocks.
extended_hours_service.py	Pre/post-market US stock prices.
bitcoin_market_service.py	BTC macro pulse (price/dominance/market cap) as context for altcoin questions.
yahoo_finance_service.py	Yahoo Finance real-time quotes (via Webshare rotating proxy).
proxy_manager.py	Webshare proxy credential/rotation management, env-var only.
marketaux_service.py	Licensed news + entity sentiment via the Marketaux API (replaces the RSS/Reddit scrapes below when a token is configured).
news_service.py	Financial news via free RSS feeds (feedparser), no API key.
sentiment_service.py	Market sentiment via the Reddit JSON API, no API key.
coinlist.py	Loads per-exchange crypto symbol lists from <code>coinlist/*.txt</code> resource files.

Environment Notes (observed this session)

File	What it does
QuantLib	Works. pip install QuantLib (or pip install "[risk]"). Powers calc_option_greeks.
LightGBM	pip install lightgbm succeeds and import lightgbm succeeds, but constructing a Booster throws FileNotFoundError loading lib_lightgbm.dll — needs the MSVC redistributable on this machine. ml_factor_service.py degrades gracefully to a pure-stdlib logistic regression without it.
stable-baselines3 / torch	Not installed, not attempted this session (multi-hundred-MB dependency chain). pip install "[rl-full]" to add it. rl_service.py's PPO path is otherwise untested — only the Q-learning fallback has been run for real.
Telegram / FRED / SEC EDGAR	All need env vars not set in this session (TV_MCP_TELEGRAM_BOT_TOKEN/TV_MCP_TELEGRAM_CHAT_ID, FRED_API_KEY, SEC_EDGAR_USER_AGENT). SEC EDGAR was still live-tested by passing user_agent directly to the function, since it needs no API key.

Everything in this document reflects code in the working tree of tradingview-mcp-custom, branch feat/ict-pine-lite-engines — local and uncommitted, per instruction not to push/pull GitHub.